MOBILE AND APPLICATION DEVELOPMENT

# Linear Layout and Its Properties


**Umar Bashir**

*Lecturer Computer Science*

android

# INTRODUCTION TO LAYOUTS

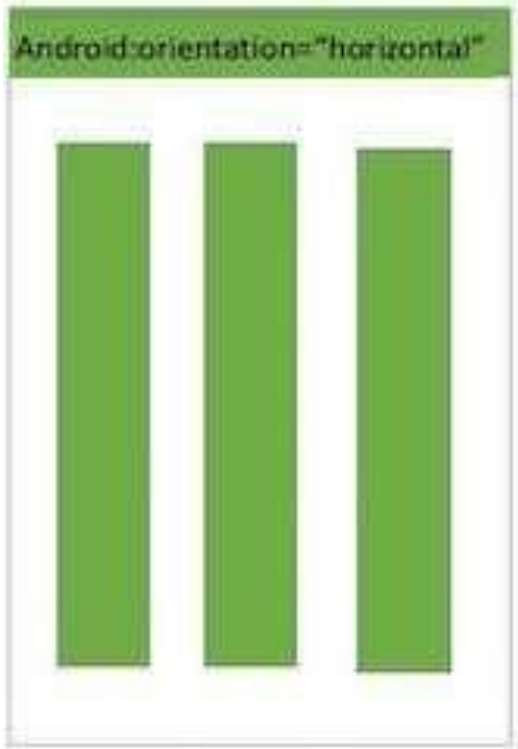
- A container is a view used to contain other views.
- Android offers a collection of view classes that act as containers for views.
- These container classes are called layouts, and as the name suggests, they decide the organization, size, and position of their children views.
- Layouts are basically containers for other items known as Views, which are displayed on the screen.
- Layouts help manage and arrange views as well.
- Layouts are defined in the form of XML files that cannot be changed by our code during runtime.

| Layout Manager | Description |
|---|---|
| LinearLayout | Organizes its children either horizontally or vertically |
| RelativeLayout | Organizes its children relative to one another or to the parent |
| AbsoluteLayout | Each child control is given a specific location within the bounds of the container |
| FrameLayout | Displays a single view; that is, the next view replaces the previous view and hence is used to dynamically change the children in the layout |
| TableLayout | Organizes its children in tabular form |
| GridLayout | Organizes its children in grid format |

The containers or layouts listed in Table 3.1 are also known as ViewGroups as one or more Views are grouped and arranged in a desired manner through them. Besides the ViewGroups shown here Android supports one more ViewGroup known as ScrollView.

# LINEAR LAYOUT

- The LinearLayout is the most basic layout, and it arranges its elements sequentially, either horizontally or vertically.




- `android:layout_width="fill_parent"`
  - The component want to display as big as its parent, and fill in the remaining spaces.
  - Ex: if parent tag has 120dp set into width & height then it will cover your whole pattern area.
  - Parents are called as main above first defining layout tag.
- `android:layout_width="match_parent"`
  - fill_parent & match_parent are the same
  - fill_parent is depricated in later Android versions.
- `android:layout_width="wrap_content"`
  - The component just want to display big enough to enclose its content only.
  - Ex: Button name as “submit”, button occupy the name size only.

# LINEAR LAYOUT (Properties)

- **android:orientation**—Used for arranging the controls in the container in horizontal or vertical order
- **android:layout_width**—Used for defining the width of a control
- **android:layout_height**—Used for defining the height of a control
- **android:padding**—Used for increasing the whitespace between the boundaries of the control and its actual content

# LINEAR LAYOUT (Properties)

- **android:layout_weight**—Used for shrinking or expanding the size of the control to consume the extra space relative to the other controls in the container. The values of the weight attribute range from 0.0 to 1.0, where 1.0 is the highest value.
- **android:gravity**—Used for aligning content within a control
- **android:layout_gravity**—Used for aligning the control within the container

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="fill_parent"
   android:layout_height="fill_parent"
   android:orientation="vertical" >

   <TextView android:id="@+id/text"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="This is a TextView" />

   <Button android:id="@+id/button"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="This is a Button" />

   <!-- More GUI components go here  -->

</LinearLayout>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</LinearLayout>
```

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:id="@+id/Apple"
        android:text="Apple"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="0.0" />
    <Button
        android:id="@+id/Mango"
        android:text="Mango"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="1.0" />
    <Button
        android:id="@+id/Banana"
        android:text="Banana"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_weight="0.0" />
</LinearLayout>
```


**Figure 3.3. (left) The `weight` attribute of the Mango `Button` control set to 1.0, (middle) the `weight` attribute of the Banana `Button` control set to 1.0, and (right) all three `Button` controls set to the same `weight` attribute**

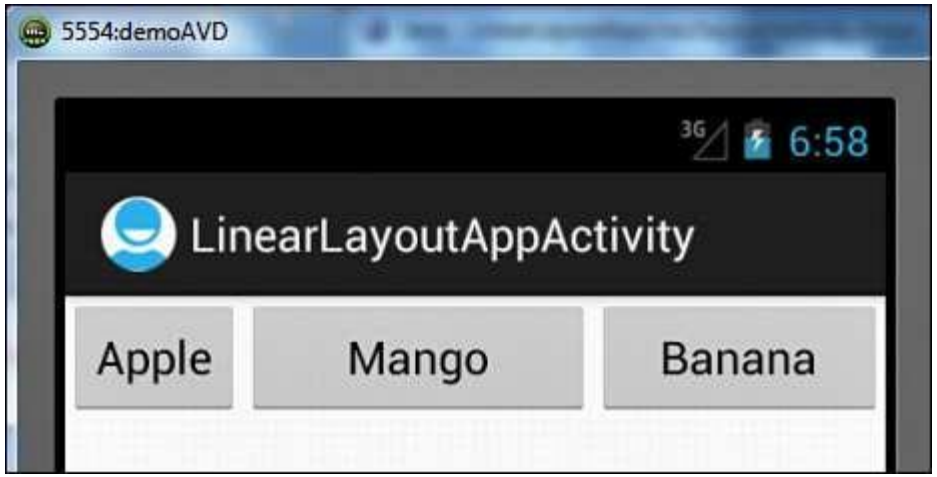



**Figure 3.4. The `weight` attribute of the Apple, Mango, and Banana `Button`controls set to 0.0, 1.0, and 0.5**

# Gravity **Attribute**

- The Gravity attribute is for aligning the content within a control.
- The valid options for android:gravity include
  - left, center, right, top, bottom, center_horizontal, center_vertical, fill_ horizontal, and fill_vertical.
- **center_vertical**—Places the object in the vertical center of its container, without changing its size
- **fill_vertical**—Grows the vertical size of the object, if needed, so it completely fills its container
- **center_horizontal**—Places the object in the horizontal center of its container, without changing its size
- **fill_horizontal**—Grows the horizontal size of the object, if needed, so it completely fills its container
- **center**—Places the object in the center of its container in both the vertical and horizontal axis, without changing its size

android:gravity="center“

android:gravity="center_horizontal|center_vertical”


**Figure 3.6. (left) The three `Button` controls vertically aligned with the `width`attribute set to `wrap_content`, (middle) the Mango and Banana `Button` controls aligned to the center and right of container, and (right) the width of the three `Button` controls expanded to take up all the available space**

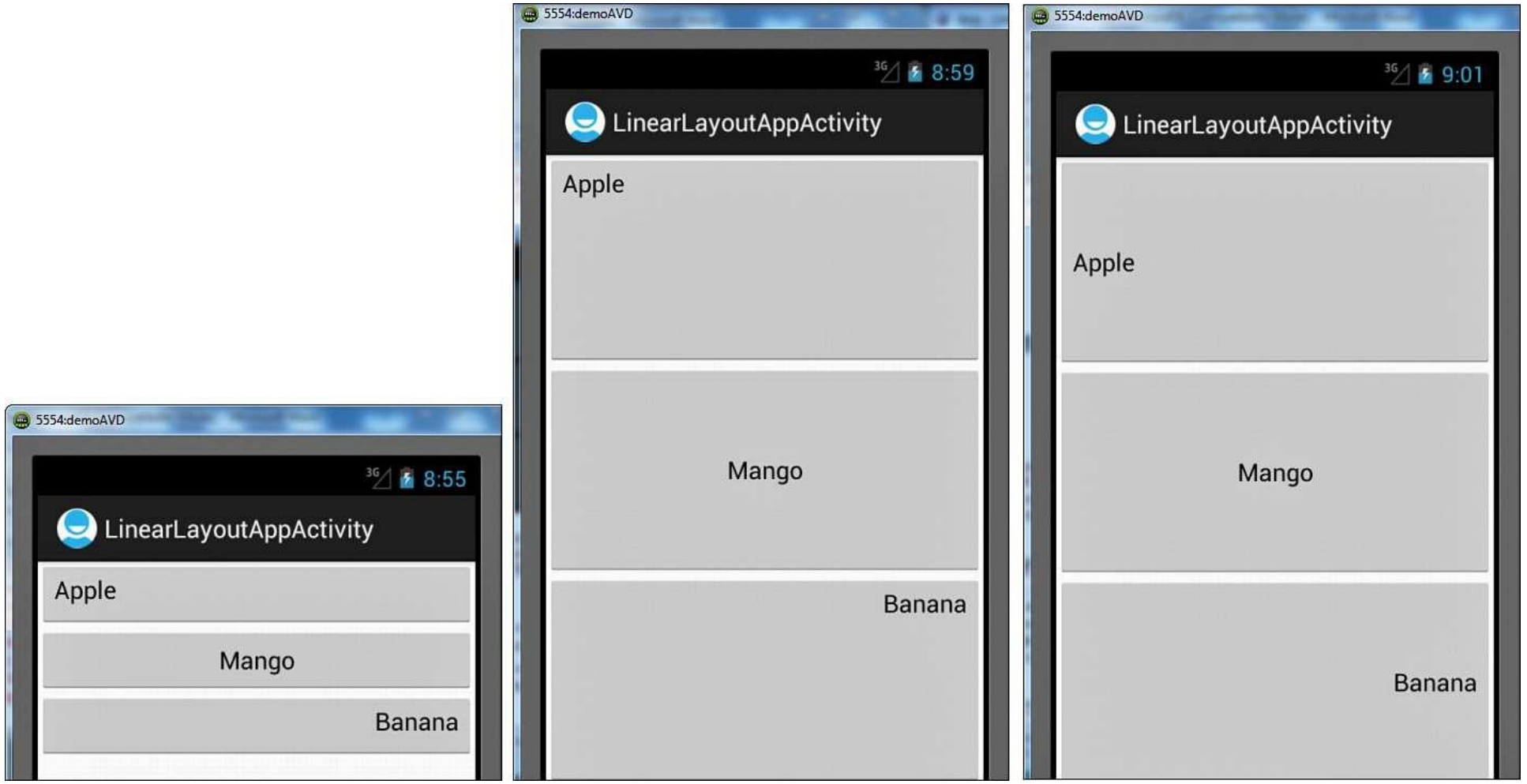
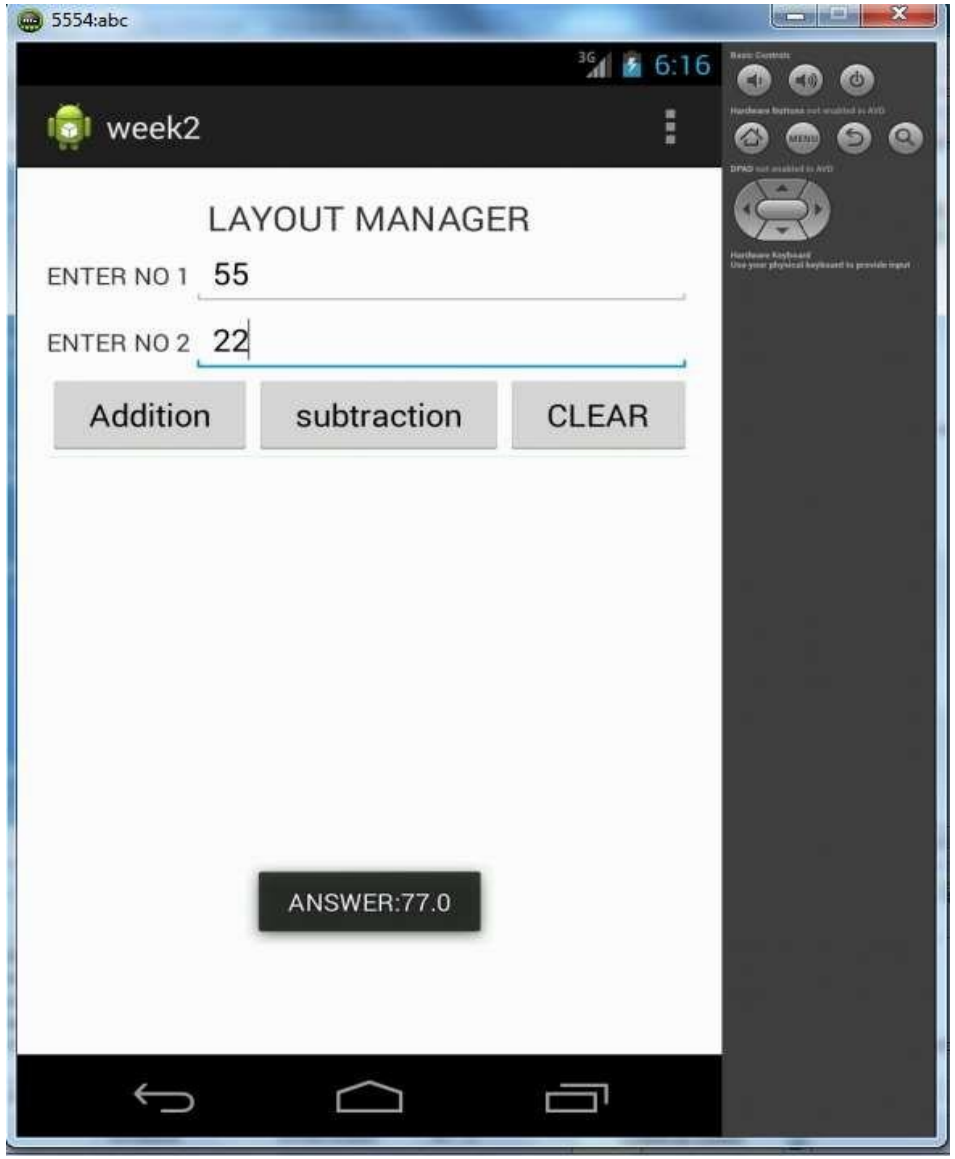



**Figure 3.7. (left) The three `Button` controls with their text aligned to the left, center, and right, (middle) the vertical available space of the container apportioned equally among the three `Button` controls, and (right) the text of the three `Button` controls vertically aligned to the center**

# Example


5554:abc
6:16
week2
LAYOUT MANAGER
ENTER NO 1
55
ENTER NO 2
22
Addition
subtraction
CLEAR
ANSWER:77.0